

FastAPI Core Concepts

1. FastAPI Project Structure

Why? Keeps a project clean and easy to maintain.

```
app/  
  main.py          # start FastAPI  
  routes/          # API endpoints  
  models/          # database models  
  schemas/         # Pydantic models  
  services/        # business logic  
  database/        # database connection  
  
  config.py        #manage .env in one file  
  
  .env             # database credential, api key and secret etc.
```

Use it when your application has multiple APIs, models, or business rules.

2. Routes

What? A route connects a URL + HTTP method to a Python function.

Why? It defines what the API should do for a request.

```
from fastapi import FastAPI  
app = FastAPI()  
  
@app.get("/employees")  
def get_employees():  
    return {"message": "All employees"}
```

Example: GET /employees calls get_employees().

3. Path Parameters

What? A value taken from the URL path.

Use it when the value identifies one resource, such as employee ID.

```
@app.get("/employees/{employee_id}")  
def get_employee(employee_id: int):  
    return {"employee_id": employee_id}
```

GET /employees/10 -> employee_id = 10

4. Query Parameters

What? Optional or filter values after ? in the URL.

Use it for search, filtering, sorting, pagination, etc.

```
@app.get("/employees")  
def get_employees(department: str | None = None):  
    return {"department": department}
```

GET /employees?department=IT

5. Request Models

What? A Pydantic model that describes data sent by the client.

Why? It validates request JSON and gives clear error messages.

```
from pydantic import BaseModel

class EmployeeCreate(BaseModel):
    name: str
    age: int

@app.post("/employees")
def create_employee(data: EmployeeCreate):
    return data

JSON:
{
    "name": "Sagar",
    "age": 22
}
```

6. Response Models

What? A model that defines the data the API should return.

Why? Keeps responses consistent and prevents unwanted fields from being returned.

```
class EmployeeResponse(BaseModel):
    id: int
    name: str

@app.get("/employees/{id}", response_model=EmployeeResponse)
def get_employee(id: int):
    return {"id": id, "name": "Sagar"}
```

7. Pydantic Validation

What? Rules that check whether input data is valid.

Use it for required fields, length, range, pattern, email, etc.

```
from pydantic import BaseModel, Field

class Employee(BaseModel):
    name: str = Field(min_length=2, max_length=50)
    age: int = Field(gt=18, lt=60)
```

Note: Pydantic validates data; database constraints such as UNIQUE should also be enforced in the database.

8. Swagger / OpenAPI

What? FastAPI automatically creates API documentation.

Why? You can test endpoints without Postman or writing frontend code.

```
Run the app, then open:
http://127.0.0.1:8000/docs
```

Swagger shows endpoints, parameters, request body, responses, and a Try it out button.

9. Dependencies

What? A dependency is reusable logic FastAPI runs before/for an endpoint.

Common uses: database sessions, authentication, current user, common headers.

```
from fastapi import Depends

def common_check():
    return "OK"

@app.get("/employees")
def get_employees(check=Depends(common_check)):
    return {"status": check}
```

FastAPI calls `common_check()` and injects its result into the route.

10. Simple Request Flow

```
Client
  ↓
Route
  ↓
Path / Query parameters
  ↓
Request model validation
  ↓
Dependency (if needed)
  ↓
Business logic / Database
  ↓
Response model
  ↓
JSON response
```

In short:

- Path parameter = identify one resource: `/employees/10`
- Query parameter = filter/search: `/employees?department=IT`
- Request model = validate input data
- Response model = control output data
- Dependency = reuse common logic
- Swagger = view and test the API